

# **CONTRACULTURA MAKER**

**RONI BANDINI**



Contracultura Maker

Roni Bandini @ronibandini

Buenos Aires, Argentina

Licencia CC BY 4.0

<https://creativecommons.org/licenses/by/4.0/>

Esta obra se ofrece bajo una licencia Creative Commons Atribución 4.0 Internacional (CC BY 4.0). Se permite compartir, copiar, adaptar y reutilizar sus contenidos, incluso con fines comerciales, siempre que se otorgue la atribución correspondiente y se indique si se realizaron modificaciones.

Primera versión 10/2024

Segunda versión 12/2025

Tercera versión 10/2026

# INTRODUCCIÓN

*“Es ineludible que cada cultura debe negociar con la tecnología, ya sea de  
manera inteligente o no”*

*Neil Postman*

*“Algunos hombres saben fabricar máquinas mágicas”*

*Alberto Laíseca*

Hay una perspectiva que nos prefiere cansados, sumisos en conformidad y viviendo un submundo algorítmico donde nuestras alegrías, nuestra economía, incluso nuestras relaciones se encuentran digitadas.

¿Qué papel juegan los dispositivos que usamos en este esquema de dominación y control? ¿Cuán funcionales son sus laberintos de uso? Y por sobre todo ¿qué empieza a suceder cuando pasamos de ser consumidores irreflexivos a creadores?

Este libro trata sobre un camino donde confluyen tecnología, contracultura, arte y activismo. Un camino que a menudo se siente como un desvío. Un camino donde el disturbio de fabricar máquinas y dispositivos que quizás no deberían existir tiene la capacidad de abrir nuevas conversaciones y cuestionar la sacralidad y el absurdo de lo establecido.

Este libro es sobre Contracultura Maker.

Roni Bandini

# MÁQUINAS, AUTÓMATAS, ROBOTS Y CYBORGS

*"Las máquinas se articulan y se definen a sí mismas frente al desorden de la  
naturaleza orgánica"*

*Anónimo*

En la Contracultura Maker fabricamos hardware ¿Por qué hardware y no software? En las últimas décadas hubo una invitación, casi un imperativo, por desarrollar software. Esta invitación llevaba asociadas promesas de posibilidades, libre albedrío y éxito. ¿Pero cuál es la agenda y el subtexto de esta invitación? En principio hay que desarmar la idea de posibilidades: El software es mayormente inofensivo cuando está confinado a un sandbox. Es decir que controlando el hardware cualquier software tiene un límite. La invitación a desarrollar software y solo software trae encubierta la imposición del terreno de batalla. ¿En qué terreno vamos a elegir librar nuestras batallas? ¿En el terreno que nos sugiere el hegemon? ¿O en uno donde, al menos por ahora, tenemos alguna posibilidad?

Por otro lado, el hardware y la manipulación de objetos físicos en general ofrece algo que el trabajo simbólico del software no puede dar. Si cortamos un cable ya no va a circular corriente. Si el tornillo no encaja, no encaja. De alguna manera el hardware es un punto de apoyo en tiempos de posverdad. En algún momento, todo maker descubre que una máquina, cualquier máquina, es difícil de ignorar, su mera existencia interpela al entorno, exige usarla, moverla, esquivarla, apagarla o destruirla. En definitiva hacemos máquinas para dejar en claro que estamos atentos al subtexto, hacemos NUESTRAS máquinas – abiertas, desprolijas, incorrectas, defensivas y subversivas – para equilibrar las OTRAS máquinas – cerradas, correctas, deshumanas, letales – y hacemos máquinas, en última instancia para tratar de comprender nuestra propia esencia.

Pero ¿de qué hablamos cuando hablamos de máquinas?

## MÁQUINAS

Una definición posible de máquina es la siguiente:

*“Objeto fabricado y compuesto por un conjunto de piezas que se usa para facilitar un trabajo determinado, generalmente transformando o aprovechando una forma de energía”*

Es muy obvio que una máquina de coser es una máquina. Porque su nombre lo dice y porque transforma fuerza cinética en costura de telas, por ejemplo. Pero la definición puede tener implicancias más sutiles, humanas y filosóficas: en la Roma antigua por ejemplo, al esclavo le decían “la máquina natural” y el arquitecto Le Corbusier decía que una casa era una “máquina de habitar”.

Las máquinas, curiosamente, no tienen por qué funcionar: pueden no hacerlo, pueden funcionar erráticamente, pueden tener principios ficcionales, pueden incluso ser conceptuales, artísticas o apagarse a sí mismas. Aun así: siguen siendo máquinas.

## MÁQUINAS INÚTILES

Este concepto fue acuñado por el artista italiano Bruno Munari. En principio quiso hacer una pieza de arte que interactuara con el entorno. Lo hizo casi como una broma en respuesta al movimiento futurista. Munari las calificó de “máquinas” porque se trató de unos colgantes hechos de partes móviles y le puso “sin propósito” porque no producían bienes para consumo material, no disminuían el trabajo manual ni incrementaban el capital.

*Quizás sirva pensar a la máquina como una perspectiva o un análisis de aprovechamiento de fuerzas que se puede hacer, tanto sobre un objeto como sujeto.*

## RELACIÓN DUAL

La relación de la máquina y el hombre siempre estuvo atravesada por cierto recelo y temor. Los Luditas eran trabajadores ingleses del Siglo XIX que sabotaban telares industriales y máquinas de hilado que amenazaban sus fuentes de trabajo. Samuel Butler decía en 1872 que era necesario destruir toda máquina no indispensable. Theodore Kaczynski, un matemático destacado de Berkeley, se fue a vivir a una cabaña en 1978 sin luz ni agua y empezó a atentar contra quienes veía responsables del avance tecnológico. Diez años antes de las acciones de sabotaje de los Luditas había sido inventado el telar de Jacquard, una máquina revolucionaria que utilizaba tarjetas perforadas para tejer patrones en la tela. Esta máquina es uno de los antecedentes más antiguos de la computación y de la utilización de bases de datos binarias. De modo que bajo un punto de vista, los Luditas quizás no

estaban destruyendo máquinas apenas por mantener sus puestos de trabajo, sino iniciando una suerte de resistencia profética ante un futuro donde las máquinas extenderían su dominación tiránica sobre los hombres.

*Los Luditas tomaron su nombre de Ned Ludd, una figura emblemática que sabotó los primeros telares. Sin embargo hasta el momento no se han encontrado pruebas concluyentes de su existencia. Como si solo hubiera estado de paso para manifestar inquietantes ecos del futuro.*

A la vez podemos ir al pasado y encontrar algunas referencias bastante opuestas sobre la relación hombre máquina. El movimiento Futurista italiano sugirió en 1909 que el concepto de belleza se traspasaba de la naturaleza y la mujer hacia la máquina. La máquina, a pesar de su amenaza latente, es la evidencia última de la pericia humana. El chauvinismo tecnológico y el status de los tecnólogos en la sociedad actual confirman que la belleza sigue estando en la máquina.

## AUTÓMATAS

Una definición posible de autómata es la siguiente: Máquina que imita figura y movimientos de un ser animado.

### Cronología

## **1000 AC**

En la biblia, libro Salmos se menciona al Golem, quizás el primer autómeta. Era una criatura animada de arcilla difícil de controlar. Ciertos Golem eran animados utilizando palabras. La palabra EMET (verdad) para activarlo y MET (muerte) para desactivarlo.

## **300 AC**

En la mitología Griega, Talos era un autómeta de bronce que rodeaba Creta tres veces por día para protegerla de los invasores.

## **1100**

Ismael Al Jazari publicó instrucciones de un autómeta para servir bebidas. Cada siete minutos y medio, las puertas de un armario se abrían para revelar el autómeta de una esclava sosteniendo una copa de vino llena y una toalla. El rey se servía el vino, se secaba la boca y el autómeta desaparecía en el armario.

## **1495**

El Caballero Armado de Da Vinci. Movía extremidades y tenía la particularidad de permitir la programación de sus movimientos mediante clavijas para alternar secuencias.

# ROBOTS

El autómeta es un ancestro del robot ya que el término robot fue acuñado por el Checo Karel Čapek recién en una obra de teatro de 1920. En esta obra de teatro los robots eran fabricados con materia orgánica, así que la palabra adquirió luego otra dimensión. En checo ROBOTA quiere decir mano de obra forzada. La palabra deriva de “esclavo”

Una pregunta que vale hacerse es ¿por qué los robots suelen ser antropomórficos? Por un lado el hombre es la medida de todo, pero también puede haber otros motivos. Como dijo E. Muslin en el libro Máquinas del Siglo XX "Una muñeca mecánica hecha con elegancia provoca una actitud cuidadosa, nadie le dará un puntapié, para nadie será agradable golpearla, digamos con un martillo."

# CARDINALIDAD

Entonces podemos identificar una línea.

1. La máquina es un aprovechamiento de fuerzas para facilitar un trabajo.
2. El autómeta es una máquina que imita movimientos de seres animados.
3. El robot es un autómeta - a menudo antropomórfico - obligado a hacer

nuestro trabajo forzado.

Pensar esta cardinalidad cuando fabricamos nuestras máquinas provoca mejores asociaciones y mayor profundidad.

## CYBORGS

El término cyborg surge de la contracción de cybernetic organism –“organismo cibernético”– y fue acuñado en el año 1960 por Manfred E. Clynes y Nathan S. Kline en el artículo Cyborgs and Space, donde imaginaron la modificación tecnológica del cuerpo humano para permitir su adaptación autónoma a las condiciones extremas del espacio. Un cyborg integra componentes orgánicos y tecnológicos en un único sistema regulado mediante información, control y retroalimentación. Así como el autómeta imita la vida y el robot hace el trabajo esclavo, el cyborg borra la frontera entre cuerpo y máquina.

*No hace falta imaginar un ser mitad hombre y mitad máquina: una persona con una prótesis o un marcapasos puede considerarse un cyborg*

# CULTURA MAKER

*“Si realmente querés entender algo, entonces lo que tenés que hacer es  
construirlo”*

*Simon Schaffer*

Vamos a empezar con una obviedad: la palabra make quiere decir hacer. Por lo tanto el maker es alguien que se define por lo que hace más que por lo que compra o usa por ejemplo.

La Cultura Maker toma lugar en la intersección entre el movimiento DIY – hágalo usted mismo -y la rama de la cultura hacker que se inclina al hardware. Entendiendo hacker con la acepción original, es decir un entusiasta de las computadoras que las hace funcionar de una forma distinta y personal.

## **PRINCIPIO**

La Cultura Maker postula un principio fundamental, muy elegante, en forma de pregunta retórica: ¿Por qué confiar en los dispositivos cuando nos proponen laberintos de uso frustrantes, nos vigilan, nos impiden usar sus capacidades al máximo y nos exigen un recambio antes de tiempo?

## **MOTIVACIONES**

La Cultura Maker puede estar motivada por la necesidad. En los años noventa la editorial cubana Verde Olivo compiló las experiencias del pueblo para

reparar y fabricar dispositivos con electrodomésticos y partes que se encontraban en la isla en un manual titulado “Con nuestros propios esfuerzos”. En Argentina la burocracia y el proteccionismo impedían ingresar electrónica y era necesario volverse ingenioso para adaptar lo existente en los procesos creativos.

A veces también la Cultura Maker puede estar motivada por una filosofía. En países desarrollados, sin impedimentos ni costos elevados, hay makers que deciden hacer aunque resulte más caro y complejo, como reacción al consumismo. Y pasa en todas partes cuando uno adquiere la mentalidad maker. Es decir cuando uno deja de ser un ente irreflexivo que acepta mansamente el dispositivo que le ponen enfrente y se asume como creador.

## ASPECTOS CLAVE

### DIY (Do It Yourself)

La práctica de hacer las cosas uno mismo en lugar de comprar o tercerizar, incluso cuando el proceso resulte más complejo o más costoso.

### Comunidad y Colaboración

El maker se apoya constantemente en el conocimiento y las prácticas



colectivas. Existe una fuerte ética de compartir los proyectos, tomando el tiempo para escribir tutoriales, comentar código, y ofrecer asistencia. Esto no quiere decir que un maker siempre deba abrir todos sus proyectos. El hecho de cerrar uno o varios proyectos por motivos contractuales o decisión personal, no lo excluye de la comunidad.

## Active Learning

Priorizar el construccionismo, la experimentación y el diseño iterativo (prototipar, fallar, corregir y mejorar) como la vía principal para adquirir conocimientos y habilidades.

## Interdisciplinariedad

La Cultura Maker borra las fronteras entre las disciplinas combinando habilidades tradicionalmente separadas, como la electrónica, la programación, el diseño, la artesanía y la mecánica, en la creación de dispositivos.

# MAKER VERSUS INVENTOR

La figura del inventor es interesante y no excluyente a otros talentos. Roberto Arlt, un gran escritor argentino, inventó medias irrompibles. Lewis Carroll, autor de Alicia en el país de las maravillas, desarrolló un ingenioso sistema para escribir en la oscuridad. La actriz Hedy Lamarr inventó el *Spread*

*Spectrum* que habilitó luego el WiFi. Sin embargo, el inventor tiene diferencias sustanciales en tanto maneja premisas con el foco puesto en patentes y comercialización. Mientras el maker “hace” sin necesidad de estar inaugurando ni de clausurarlo la posibilidad a otros de crear algo parecido o incluso exactamente lo mismo.

## TÉRMINO MAKER

El término maker surge alrededor del año 2005 cuando coinciden varios acontecimientos: La placa Arduino se lanza ese año, desarrollada por Massimo Banzi y el equipo del Interaction Design Institute Ivrea. Al mismo tiempo sale la revista Make, fundada por Dale Dougherty. Un año más tarde se organiza la primera Maker Faire en San Mateo, California y se lanza el proyecto RepRap de Adrian Bowyer para impresión 3D.

*No obstante, Makers hubo siempre, solo que sin ese rótulo. Por mencionar algunos: Juan Baigorri Velar en Argentina con la máquina para detección mineral de 1938, Don Lancaster en Estados Unidos con el TV Typewriter de 1973, Clive Sinclair en Inglaterra con la Sinclair MK14 de 1977.*

En cuanto a publicaciones, la revista argentina Lúpín de 1966 era una publicación maker, así como la Popular Electronics norteamericana de 1954,

la Everyday Electronics inglesa de 1971, o la Elektor de Países Bajos en 1961.

# CONTRACULTURA MAKER

*“Lo que se necesita son personas que puedan volverse más extrañas que el  
extraño mundo que hemos producido”*

*John Durham Peters*

La Contracultura es la oposición y el cuestionamiento a las normas y valores sociales mayoritarios. Suele asociarse a los Estados Unidos de los sesenta, pero en realidad se aplica también a otras circunstancias similares como la Bohemia francesa del 1600. La Contracultura implica el rechazo y la crítica a lo establecido, que en términos tecnológicos y de inclusión de la Inteligencia Artificial, adquiere nuevas escalas y significados.

El término Contracultura, tan en auge durante los años sesenta y setenta, parece haber sido devaluado. Resulta que en ciertos ambientes académicos se planteó que a partir de la posmodernidad no puede existir nada fuera de la cultura o contra la cultura, y si bien esto tiene sentido para un ensayo, es una perspectiva algo siniestra en tanto pretende instalar que no hay pelea posible.

*Muchos de nosotros creemos que sigue existiendo la posibilidad de una Contracultura y no solo eso, sino que es más necesaria que nunca.*

## CONSTRUCCIÓN

La Contracultura Maker se construye sobre la Cultura Maker. Tiene sus mismos principios, dinámicas y protocolos, pero le agrega una restricción y una liberación. La restricción es que tenemos que intentar oponernos a lo establecido con cada máquina o dispositivo.

Este intento a veces es muy claro en la función del dispositivo. Por ejemplo los anteojos para evitar reconocimiento facial, inspirados en el caso Ibarrola pero en otras ocasiones es también algo de un orden más sutil y conceptual: el hecho de “desperdiciar” recursos que la industria necesita y está muy dispuesta a pagar, en alguna otra dirección o bien en subvertir el uso estándar de un dispositivo llamando la atención sobre eufemismos o intencionalidades ocultas.

*En agosto de 2019 Guillermo Ibarrola fue identificado por un sistema de reconocimiento facial ruso contratado por el gobierno de la Ciudad de Buenos Aires. Estuvo preso varios días hasta que se advirtió el error.*

En cuanto a la liberación, la Contracultura Maker nos libera de la necesidad de buscar una utilidad consensuada en los dispositivos. Por ejemplo, es fácil encontrar consenso en que agregando motor a un skate el dispositivo resultante es útil, pero seguramente costará encontrar consenso en la utilidad de una máquina para hacer llover, que en efecto no logra hacer llover.

*La pregunta inevitable es: ¿por qué hacer una máquina que no cumple su función? Para investigar, para colaborar, para llamar la atención sobre un tema – la pluvicultura por ejemplo –, para rescatar o re-visitar una historia, como arte electrónico de hiper complejidad enterrada y en última instancia esa holgura entre el nombre de la máquina y su función declarada puede estar llena de significado.*

## DETOURNEMENT

Ahora bien ¿por qué la cultura fomenta los estudios tecnológicos cuando ahí, cada vez más, se encuentra el germen para disputar lo establecido? Por un lado estos estudios tecnológicos tienen un marco de especialización y especificidad elevado y por otro, quienes los reciben suelen estar demasiado endeudados y artificialmente desesperados como para explorar las intersecciones entre dominios del conocimiento tecnológico o potenciales *detournements* sin recompensa inmediata.

*Detournement refiere a tomar palabras, ideas y objetos para darles un uso distinto.*

Quizás uno es un maker cuando aprovecha el flujo del consenso y pasa a la contracultura maker cuando su “hacer” tiene la intención explícita de abrir una puerta de extrañeza, generar el disturbio en lo establecido y habilitar conversaciones improbables.

## TRANSITAR LA INCERTIDUMBRE

En el proceso de Contracultura Maker, entre la ignorancia de partida y la comprensión a menudo existe un espacio liminal donde las explicaciones anteriores no alcanzan y las nuevas todavía no se estabilizaron. Al transitar este espacio es necesario administrar estados de confusión y frustración, así como desarrollar tolerancia ante la ambigüedad sin pausar ni detener el hacer. Construir al margen implica trabajar con documentación incompleta, componentes desconocidos, resultados impredecibles y problemas que quizás no encuentren solución.

En algún momento empezamos a internalizar una suerte de *Fuzzy Logic* donde reemplazamos las clasificaciones binarias por grados de pertenencia. En lugar de pasar de “no saber” a “saber” podemos entender un concepto en cierto grado, aplicarlo en algunos contextos y construir, aún sin comprender de momento todos sus fundamentos y posibilidades.

*El conocimiento en la Contracultura Maker suele comenzar parcial y provisional, como una intuición operativa y mediante pruebas, errores, y observaciones, esa intuición va ganando precisión.*

Una analogía válida es animarse a ingresar a un territorio con un mapa incompleto confiando en que vamos a encontrar pistas, ayuda en la comunidad y que eventualmente vamos a desarrollar mecanismos propios para orientarnos.

# ACTIVISMO

En tanto activismo la Contracultura Maker es un camino de resistencia que permite negar la credibilidad de esa apariencia donde no hay alternativa salvo someterse mansamente a la brutalidad de la tecnología. Es usar la tecnología para combatir la misma tecnología cuando oprime la humanidad, las posibilidades creativas y cuando reduce las alternativas a una serie de opciones igualmente frustrantes.

# MEETUPS

Bajo la perspectiva del control el experimento del encierro fue exitoso. Cada vez más gente trabaja en su casa y permanece ahí sintiendo que salir es inconveniente, caro y arriesgado. La obviedad es que cuando la comunicación con los demás está limitada a las redes somos más vulnerables. Encontrarse, descubrir a la persona detrás de su “persona” en las redes y compartir un espacio tiene un beneficio especial: descartamos una parte de la impostura y lo simulable.

Si bien encontrarse no necesita más excusa que una obsesión compartida o una actividad divertida, van algunos formatos probados y convocantes dentro de la Contracultura Maker:

## **Meetups de reparación**

Hablar de obsolescencia, recuperar y extender la vida útil de dispositivos electrónicos. Aprender a diagnosticar, soldar y re aprovechar partes de dispositivos.

## **Meetups de privacidad**

Encuentros para explicar cómo usar TOR, PGP, cifrado, gestores de contraseñas, firewalls, comunicaciones seguras y reducción de las huellas digitales

## **Meetups maker sin Internet ni IA**

Encuentros para desarrollar dispositivos sin acceder a Internet ni usar LLMs

## **Meetups de música DIY**

Encuentros y muestras de Circuit Bending, MIDI, síntesis y efectos.

## **Meetups de tecnología obsoleta**

Encuentros para explicar, probar e integrar electrónica supuestamente obsoleta

## Meetups de soberanía tecnológica

Encuentros para exponer qué tecnologías fueron propuestas en zonas periféricas. Qué las hizo triunfar o fracasar. Qué se puede proponer hoy contemplando las necesidades y particularidades de una región.

# RAZÓN INSTRUMENTAL

La razón instrumental es un concepto desarrollado por la Escuela de Frankfurt y se define como:

*la eficiencia de los medios para alcanzar objetivo predeterminado, sin cuestionar la racionalidad o el valor del fin en sí mismo*

Es algo que tenemos muy internalizado en esta modernidad y que se puede visualizar como el camino más corto entre un problema y su solución.

El culto a la eficiencia económica y la funcionalidad nos lleva a repetir los caminos más transitados, impide la creatividad y nos reduce a medios para el aparato productivo.

*Para la Contracultura Maker, más importante que dominar tecnologías específicas, es liberarse de la razón instrumental.*

Es decir, emanciparse del “yo default” en tanto engranaje de una máquina tan grande que nadie sabe para qué sirve. Esta liberación es una diferencia sutil pero fundamental entre jugar con una placa Arduino que va a quedar finalmente al fondo de un cajón versus hacer, como un descubrimiento, una identidad y un acto de resistencia.

## ARTE ELECTRÓNICO

Si bien hay dispositivos de Contracultura Maker indistinguibles del arte electrónico y arte electrónico que tranquilamente podría ser Contracultura Maker, la diferencia no reside en la estética ni en la tecnología utilizada, sino en las condiciones de origen y de validación.

En términos generales, los dispositivos de Contracultura Maker no surgen pensados para la exhibición. No responden a las lógicas del montaje, la circulación institucional ni la lectura curatorial. Suelen nacer como soluciones personales, experimentos técnicos, pruebas de concepto o incluso como artefactos obsesivos, donde la función, la exploración o el desafío técnico preceden a cualquier consideración expositiva. Esto se traduce en una característica recurrente: grados de complejidad excesivos o desalineados con el resultado visible. Hay más código del necesario, más electrónica de la que el efecto final justificaría, más ingeniería que narrativa.

En el arte electrónico, incluso en sus vertientes más experimentales, la complejidad tiende a estar domesticada por la experiencia del espectador.

*En la Contracultura Maker, en cambio, la complejidad suele estar orientada hacia adentro: satisface al maker, documenta un proceso, explora un límite técnico o resuelve un problema sin consenso.*

## TEMPLATES MAKER

Un maker es alguien que hace, no alguien colgado en un loop eterna mientras piensa qué hacer. Sin embargo la creatividad requiere ciertas condiciones contrarias a esta modernidad. Es necesario tener holgura en las rutinas, es necesario tener valentía para emprender caminos que podrían no resultar productivos, es necesario emanciparse de la razón instrumental y es necesario tener paciencia ya que la creatividad es una práctica. Mientras se modifican los aspectos personales para habilitar la creatividad, la Contracultura Maker propone una serie de templates para identificar ideas y ponerse a trabajar.

### El método Hunter

Hunter S. Thompson fue un escritor y periodista, creador del periodismo Gonzo. Cuando quiso tener la sensación de escribir una gran novela agarró El Gran Gatsby y lo tipeo letra a letra y palabra a palabra. Del mismo modo

vamos a agarrar un proyecto bien documentado y lo vamos a replicar con el ánimo de que podamos apropiarnos en alguna instancia modificando componentes o aspectos funcionales y si esto no sucede, igual habremos ganado pericia y experiencia.

## Crossover

Se trata de tomar algo que existe en digital y pasarlo a físico o bien tomar algo mencionado en la ficción y pasarlo a real. En este último caso podría suceder que los principios ficcionales se arrastren, pero mientras no se oculte el hecho, no existe ningún problema en avanzar con el dispositivo. Un ejemplo de este template es La máquina de Klausner. Inspirada en el cuento La máquina del sonido de Roald Dahl, este dispositivo lee las condiciones de humedad de la tierra y mapea las mediciones con audios.

*A veces parecemos olvidar que la ciencia tiene más que ver con descubrimientos que con hechos y la ficción a menudo es capaz de anticipar estos descubrimientos o indicar hacia dónde buscar. El cuento de Roald Dahl y el trabajo de Jagadish Chandra Bose inspiraron a un interrogador de la CIA llamado Cleve Backster, quien aplicó el polígrafo a las plantas en 1968. Sus métodos fueron objetados por la ciencia. Sin embargo los profesores Yossi Yovel y Lilach Hadany de la Universidad de Tel Aviv lograron demostrar con ayuda de Machine Learning que las plantas en efecto emiten mensajes cuando se encuentran estresadas.*

## Upcycling

Es una variante del reciclado donde se toma un dispositivo y se fabrica con él un dispositivo superador. Por ejemplo la punta de un soldador y el motor de rotación lenta de un horno eléctrico se usan para fabricar un micro-shawarma de escritorio.

## Extravagancia

Un problema modesto se resuelve con extravagancia de recursos. A modo de ejemplo: la necesidad de mantener despejado el escalón de acceso a una vivienda resulta en un sistema cliente servidor con reconocimiento vía Machine Learning, comunicación Bluetooth y emisión de alertas progresivas.

## Autonomía con IA

Ante un problema determinado se desarrolla una máquina autónoma con IA. Un ejemplo de este template es el dispositivo Reggaeton Be Gone que identifica cierto estilo musical con Machine Learning e intenta desconectar el emisor o deteriorar la calidad del sonido.

# VS

En nuestra existencia digital a menudo creemos que encontramos algo o que llegamos a ese algo de casualidad cuando en realidad fue puesto ahí para

interceptarnos. Somos en definitiva una suma de patrones y las plataformas, con ayuda del Machine Learning, explotan esos patrones para dirigirnos en una u otra dirección, limitando nuestros caminos naturales, nuestros procesos y posibilidades creativas. VS (Volquete Surprise) es una actividad que surgió en los encuentros de Contracultura Maker para recuperar el factor azaroso de los descubrimientos.

El procedimiento es sencillo: consiste en tomar caminos azarosos por zonas poco habituales, atentos a los descartes tecnológicos al costado de los volquetes. Una vieja impresora, un walkman o un container lleno de lámparas de camping pueden ser la materia prima, la inspiración o el *rabbit hole* hacia el próximo proyecto.

## PEDIR AYUDA

Si bien la Contracultura Maker es colaborativa: redactamos tutoriales, compartimos el código y ayudamos a los demás, hay formas de lograr mejores resultados en esta ayuda. Esto se aprende a la larga, pero mejor es ahorrar camino.

Antes de contactar al autor de un proyecto, es necesario recopilar información: qué placa, qué versión de software y cuál es el error concreto por ejemplo.

También sirve buscar el error y posibles soluciones. Quizás el autor ya respondió eso varias veces o bien un problema similar ya resuelto puede evitar darle más trabajo.

*Contactar en privado es siempre el último recurso. Si el autor subió el proyecto a una plataforma con mensajería pública, se debe escribir ahí.*

En el mensaje, además de la recopilación de información intentar alguna hipótesis, referir alguna prueba infructuosa, algo que refleje esfuerzo por resolver el problema.

Por último, nunca olvidar alguna apreciación por el trabajo ya realizado. Después de todo el autor seguramente ayude, pero no está obligado a hacerlo.

*¿Qué sentido tiene consultar a un maker cuando es posible usar la IA generativa? Dejando de lado que algunos “makers” usan íntegramente la generativa sin comprender nada de lo publicado - y quizás la IA es el lugar a recurrir para esos casos – consultar al maker genera devoluciones, aclaraciones y mueve el proyecto hacia alguna dirección humanamente positiva. Por otro lado hay contenidos demasiado marginales, para los cuales la IA generativa quizás no tenga respuesta satisfactoria en cualquier caso.*

# INTELIGENCIA ARTIFICIAL

*“La gente teme que las computadoras se vuelvan demasiado inteligentes y se apoderen del mundo, pero el verdadero problema es que son demasiado estúpidas y ya se han apoderado del mundo”*

*Pedro Domingos*

Nos hacemos llamar *homo sapiens* (hombre sabio), de modo que la inteligencia tiene que tener un lugar de importancia en nuestra especie y algo llamado Inteligencia Artificial pone en disputa nuestras asunciones, nuestras funciones y relaciones.

¿Pero de qué habla la gente cuando habla de IA? Incluso para quienes trabajan con tecnología a veces no queda muy en claro. Para definir IA, podría ser útil definir inteligencia, lo cual no es tan sencillo.

## **Inteligencia**

¿Qué significa ser inteligente? Un acercamiento intuitivo puede relacionarlo a la capacidad de pensar. Sin embargo hay organismos como el moho de fango que encuentran la salida de un laberinto sin tener un sistema nervioso central, es decir: sin tener cerebro. Vamos a decir entonces que inteligencia es “hacer lo correcto en el momento adecuado”

## **Inteligencia artificial**

Ahora ¿Qué es la “inteligencia artificial”? Existen muchas perspectivas y definiciones. A estos efectos vamos a decir que la inteligencia artificial es un sistema artificial que toma decisiones inteligentes basadas en algún tipo

de entrada. Ejemplo: frenar un vehículo autónomo cuando la distancia a un obstáculo es menor a X centímetros.

En esta visión de inteligencia suele haber una precondition (si la distancia es menor a X) y una acción (frenar)

## ¿Es la IA inteligencia?

*"Preguntar si las máquinas pueden pensar es tan relevante como preguntar si los submarinos pueden nadar"*  
Edsger Dijkstra, 1984

En 1770 Von Kémpelen inventó el Mechanical Turk, un autómeta inteligente para jugar al ajedrez con el que ganó partidas por todo Europa. Luego se descubrió que era un fraude:

tenía una persona de talla pequeña adentro jugando con un ingenioso sistema de poleas y espejos. Muchos años después del Mechanical Turk, IBM fue capaz de crear una máquina real llamada Deep Blue, capaz de ganarle a cualquier campeón de ajedrez. En ese punto los críticos empezaron a decir que ganar al ajedrez no era representativo de inteligencia. Este fenómeno lleva por nombre el "Efecto IA".

*Cada vez que los investigadores de Inteligencia Artificial encuentran la*

*forma para que una computadora lleve a cabo una tarea, los críticos dicen que esa tarea no es representativa de inteligencia.*

Abarcar el concepto de inteligencia es inconducente porque el ser humano, amenazado, lo va moviendo. En cualquier caso, desde la tecnología el interés no está puesto en “crear máquinas que piensen como hombres” sino de permitir a las computadoras llevar a cabo tareas que solían requerir inteligencia humana.

## **IA débil y fuerte**

Desde la filosofía existe una división de la IA entre débil y fuerte. Débil quiere decir que las máquinas pueden actuar como si fueran inteligentes. IA fuerte quiere decir que las máquinas efectivamente son inteligentes y están pensando.

## **Machine Learning**

Ya en el campo de sus aplicaciones, la IA puede ser considerada como un paraguas que tiene muchas cosas debajo, una de ellas y no la más comprendida, es el Machine Learning.

*El Machine Learning es una forma de descubrir patrones sobre cómo funciona el mundo. El Machine Learning aprende de datos históricos, sin necesidad de ser programado específicamente.*

Usamos Machine Learning cuando por ejemplo: le enseñamos a un dispositivo a reconocer caras en una foto o sonidos o combinaciones de gases. ¿Por qué no usar programación tradicional heurística? En un caso hipotético es necesario detectar si un cuarto está ordenado o desordenado. Sacamos fotos del cuarto desordenado en muchas formas distintas. Luego del cuarto ordenado en muchas formas distintas. Ya con las fotos tomadas y clasificadas, analizar las diferencias va a requerir un experto en imágenes y muchísimas reglas. Y después de todo el tiempo y el dinero invertido, el sistema no va a funcionar bien. Esto mismo es muy sencillo de resolver con Machine Learning. Se sacan muchas fotos del cuarto ordenado, muchas del cuarto desordenado. Y desde este punto el Machine Learning se encarga de encontrar los patrones automáticamente. Podemos incluso probar si el entrenamiento es exitoso mostrándole fotos que no vio hasta el momento y determinando el porcentaje de detecciones correctas e incorrectas.

## Datos en Machine Learning

En el libro *The Master Algorithm* de Pedro Domingos se expresa esta idea de que las computadoras habilitaron Internet, Internet habilitó la captura de datos a gran escala y la captura de datos a gran escala habilitó el Machine Learning. Los datos son entonces el ladrillo en la construcción del Machine Learning y esto les otorga una responsabilidad especial.

Compilar un set de datos es típicamente la parte más complicada, cara y

lenta de cualquier proyecto. Es también el punto donde se pueden cometer errores complicados de detectar que van a contaminar el proyecto entero.

Los sets de datos tienen que tener ciertas características:

**Relevantes:** los set de datos deben contener información útil para el problema a resolver. Ejemplo: en un sistema que usa el ritmo cardíaco para estimar el rendimiento atlético, ese dataset tiene que tener datos del sensor cardíaco y no un sensor de humedad en tierra.

**Representativos:** los sets de datos deben tener información sobre todas las condiciones. A modo de ejemplo en un kiosko de renovación de pasaportes en Australia no contemplaron la posibilidad de la ascendencia oriental de los ciudadanos y esto les impedía sacarse las fotos ya que el kiosko detectaba que tenían los ojos cerrados.

**Balanceados:** Si bien existen técnicas de Data Augmentation, en líneas generales si tengo los datos demasiado desbalanceados los resultados no van a ser satisfactorios. Esto pasó por ejemplo con Google Photos y el escándalo de la etiqueta “gorilas”

**Confiables:** Un set de datos debe ser consistente. Si contiene errores, deben ser uniformes. Si hay ruido debe ser del mismo tipo y magnitud presente en el mundo real. Por ejemplo, si en un proyecto me quedo sin sensores y encargo la mitad un fabricante diferente con calidad inferior, eso

podría generar ruidos que contaminen el proyecto.

## Overfit

El overfit es el efecto de sobre-entrenar un algoritmo de aprendizaje al punto donde encuentra un patrón que no es verdad en el mundo real.

## IA generativa

Si bien el Machine Learning tiene aplicaciones prácticas útiles y sólidas, muchas veces pasa inadvertida salvo para los especialistas. Por su parte la Inteligencia Artificial generativa concentra toda la atención. Cualquiera puede escribir un prompt y obtener “mágicamente” texto, música, código e imágenes.

La IA generativa es entonces una rama de la IA – mal pensada a veces como la totalidad de la IA - que se enfoca en generar contenido original a partir de datos existentes. Los modelos de IAG aprenden patrones y estructuras en los datos de entrenamiento y generan nuevos datos con características similares.

Los resultados de la IA generativa son a menudo tan sorprendentes que pueden llevar a confusiones con respecto a su naturaleza. Confusiones que impulsan a asignarle características que sencillamente no posee. Una forma de resistir este efecto es pensar la IA generativa como un sistema

predictivo sofisticado. Del mismo modo que el textbox de un buscador va mostrando la palabra siguiente más probable.

## LLM

En el centro de la IAG se encuentran los LLM. Un LLM es un sistema avanzado de inteligencia artificial que se especializa en procesar, entender y generar texto similar al humano. Estos sistemas se suelen implementar como una red neuronal entrenada con enormes cantidades de texto, lo cual les permite aprender patrones complejos del lenguaje y les da la capacidad de realizar distintas tareas como traducción automática, generación creativa de textos, respuestas a preguntas, resumen de textos, y otras tareas relacionadas al razonamiento y el lenguaje.

Estas tareas tan espectaculares son llevadas a cabo con predicciones. Por ejemplo, ante el prefijo “La ciudad más famosa en Estados Unidos es...”, el LLM seguramente podrá dar una alta probabilidad a “Nueva York” y “Los Angeles” y una baja probabilidad a “manzana” y “laptop”.

Es posible crear un LM básico por medio de una tabla Ngram. Las tablas Ngram están basadas en los modelos probabilísticos para analizar el lenguaje de Claude Shannon. Un modelo de Ngram predice  $X_i$ , basado en  $X_{i-1}$ ,  $X_{i-2}$ , etc donde se suele asumir que  $X_i$  solo depende de los últimos  $n$  valores de la secuencia para ahorrar recursos computacionales. En el caso de  $n=1$  el Ngram se llama Unigram y es tan sencillo como la siguiente

tabla:

```
"Hoy" 0.2  
"Micrófono" 0.05  
etc
```

## Transformers

Un LM más moderno estará basado en transformers: un tipo de red neuronal que puede procesar secuencias de tokens en paralelo gracias a un mecanismo de "self-attention". Esta tecnología fue desarrollada por Google en 2017 para la traducción de textos y es un modelo de secuencia-a-secuencia. Es decir que convierte secuencias de un dominio a otro dominio. Por ejemplo, una frase del inglés al francés. La arquitectura original de los transformers consiste en dos partes: un encoder y un decoder. El encoder convierte el texto de entrada – la frase en inglés - en una representación que luego es enviada al decoder. El decoder usa esa representación para generar el texto de salida autoregresivamente.

## Pre-training

Por su parte el pre-training es la etapa fundacional donde un LLM se entrena con sets de datos masivos, diversos y sin etiquetar. La tarea acá es

predecir el siguiente token dado un contexto previo. Esta etapa es lenta y cara. Después del pre-training es posible llevar a cabo el fine-tuning ya más rápido y económico. Se usan datasets específicos con ejemplos positivos para ciertas tareas.

## Reinforcement

Tras el fine-tuning es posible realizar una segunda capa del mismo proceso llamada reinforcement learning from human feedback o RLHF. En esta etapa es donde por ejemplo se incorporan filtros de seguridad para evitar que el LLM responda de manera políticamente incorrecta. A diferencia del paso anterior, acá se pueden usar ejemplos negativos.

Profundizando el nivel de detalle queda claro que los LLM son sistemas predictivos sofisticados. Entonces ¿de dónde viene la asignación de cualidades extra? Por un lado los LLM suelen mostrar “comportamientos emergentes”, es decir la habilidad de resolver problemas para los cuales no fueron específicamente entrenados.

Por otro lado existe algo llamado El efecto Eliza.

*El efecto Eliza se puede definir como la tendencia a asumir, inconscientemente, que los comportamientos informáticos son análogos*

*a los comportamientos humanos.*

El efecto Eliza surgió en 1966 a partir del bot conversacional ELIZA, desarrollado por el informático del MIT Joseph Weizenbaum.

## Contexto y búsqueda vectorial

Una confusión, quizás inducida, es asumir que un LLM aprende con nuestras interacciones y llega a “conocernos”. En realidad, una conversación no modifica los parámetros del modelo. Durante una sesión, el sistema puede utilizar el contenido previo como contexto y, dependiendo de la aplicación, también puede disponer de algún mecanismo externo de memoria que recupere información relevante sobre las interacciones anteriores. Esa información sencillamente se agrega al contexto antes de generar una nueva respuesta.

*Una analogía posible es que nos presenten a una persona, conversemos todo el día y al día siguiente esa persona no recuerde quiénes somos ni qué hablamos, salvo que tenga algún tipo de memoria externa: un cuaderno, una base de datos o algún otro registro que pueda consultar.*

Si bien una búsqueda tradicional de la época pre-LLM podía utilizar técnicas como índices invertidos, stemming y TF-IDF para encontrar documentos relevantes, seguía dependiendo de la relación entre los

términos de la consulta y los términos en los documentos. Por ejemplo, una búsqueda por “automóvil” podía tener problemas para recuperar un email que mencionaba la palabra “carro”.

La búsqueda vectorial introduce otro enfoque. Primero, un modelo de embeddings transforma textos –palabras, frases, documentos o consultas completas– en vectores numéricos dentro de un espacio de muchas dimensiones. De esta forma los textos que representan conceptos relacionados tienden a quedar próximos en ese espacio. Ante una consulta, el sistema genera su vector y lo compara con los vectores almacenados utilizando alguna medida de similitud o distancia, como la similitud de coseno o el producto escalar.

*Así el sistema transforma el problema de “¿esta palabra aparece?” en algo más del tipo a “¿cuán cerca se encuentra este contenido del concepto que estoy buscando?”*

Un email que habla de “comprar un vehículo nuevo” se va a mostrar como relevante para una consulta sobre “automóviles”, aunque ninguna de las dos expresiones coincida exactamente.

Este mecanismo es una de las piezas fundamentales de los RAG (Retrieval-Augmented Generation): primero se recupera información relevante mediante búsqueda –en general vectorial– y luego esa información se incorpora al contexto que recibe el LLM.

Con este truco, los modelos de lenguaje potencian la idea de que no solo son capaces de localizar contenido por un keyword, sino que comprenden conceptos o similitudes no tan obvias.

## **Cuándo empezó la IA**

Está instalada la versión de que la IA es una revolución situada alrededor del año 2022 con la aparición de chatGPT, pero es posible ir hacia atrás y encontrar:

2017: paper del modelo transformer

1997: Deep blue derrota a Kasparov

1986: se desarrolla el algoritmo de back propagation

1958: se inventa el Perceptrón

1956: se acuña el término IA en la conferencia de Dartmouth

1911, Frederick Winslow Taylor, publica principios de la administración científica

## **Riesgos de la IA**

La IA se suele presentar como la solución a todos los problemas cuando también agrega sus propios problemas, a veces de un orden más sutil y no

por eso despreciable. La IA es además una especie de campo de batalla y en toda batalla hay tantos engaños y estratagemas como vencidos y vencedores.

**Riesgo nominal:** la Inteligencia Artificial se llama así, pero bien podría haberse llamado de otra manera: algoritmo de inferencia por clasificación o generación predictiva de difusión estable en algunas de sus ramas por ejemplo. Por algún extraño motivo, la gente suele ser demasiado literal con la Inteligencia Artificial. Como si apenas por el hecho de llevar la palabra inteligencia toda salida de un sistema de IA fuera necesariamente inteligente. Un sistema de Machine Learning entrenado con datos desbalanceados por ejemplo, podría dar una salida absurda.

**Riesgo en los datos:** en el año 2015 Google implementó una característica novedosa en Google Photos: el etiquetado automático con IA. Unos chicos afroamericanos vieron que tenían la etiqueta “gorilas”, lo cual les llamó la atención porque no habían estado en un zoológico y resultó que eran fotos de ellos mismos. Según parece ciertas categorías estaban sub-representadas.

**Riesgos humanos:** en 2019 un sistema de reconocimiento facial ruso indicó a Guillermo Ibarrola como prófugo de la justicia. Ibarrola exclamaba no tener nada que ver con el suceso, pero se confió en lo que decía la IA y lo metieron preso. Luego se descubrió que un empleado encargado de la carga de los datos había equivocado el segundo nombre del sospechoso.

**Riesgo de explicación:** en Machine Learning la detección de patrones se hace automáticamente. Esto presenta un carácter de “caja negra” que impide explicar por qué se tomó tal o cual decisión.

**Riesgo de sim2real:** así como existen los datos, existen los datos sintéticos. Los datos sintéticos no son obtenidos de forma natural, sino simulados o incluso generados con IA generativa. Estos datos sintéticos, a menudo indistinguibles de los reales, se utilizan para entrenar modelos o testearlos con loops y efectos impredecibles.

# ELECTRÓNICA

La electrónica es fascinante y la base de las computadoras. Entender sus fundamentos permite ganar profundidad en la comprensión de la tecnología así como en la fabricación de dispositivos.

## Conductor y aislante

Un material que tiene alta resistencia a la electricidad se conoce como aislante. La mayoría de los plásticos son aislantes. Un material con baja resistencia es un conductor. Los metales como el cobre, el aluminio, la plata y el oro son buenos conductores.

## Medidas

La presión eléctrica es medida en volts. El flujo eléctrico es medido en amperes. Y la resistencia en Ohm.

Ley de Ohm

volts = ohms × amps

$$V = I \times R$$

$$I = V/R$$

$$R = V/I$$

V es la diferencia de voltaje entre dos puntos de un circuito simple. I es la corriente en amperios fluyendo a través del circuito entre dos puntos. Se usa la I porque antes se usaba “inductance” R es la resistencia en ohms entre dos puntos.

La ley de Ohm es de utilidad, por ejemplo para determinar si un componente se va a quemar.

## Watts

Un watt es una unidad de trabajo. Los ingenieros tienen su propia definición de trabajo, dicen que un trabajo es realizado cuando una persona, un animal o una máquina empuja algo para superar la resistencia mecánica.

Cuando los electrones se abren camino por un circuito, superan una especie de resistencia y por lo tanto llevan a cabo un trabajo que puede ser medido en watts. Las lámparas y la amplificación por ejemplo se miden en watts.

$$\text{watts} = \text{volts} \times \text{amps}$$

Usando los símbolos estándar

$$W = V \times I$$

$$V = W/I$$

$$I = W/V$$

## Serie y paralelo

Cuando se conectan resistencias en paralelo, la resistencia se suma. Cuando se conectan en paralelo la resistencia se divide. Cuando se conectan dos baterías en serie se dobla el voltaje. Cuando se conectan dos baterías en paralelo dan el mismo voltaje por el doble de tiempo.

## Corriente continua y alterna

La corriente de una batería se llama corriente continua (CC). Es un flujo constante en una sola dirección, similar al flujo de agua de una canilla. Por otro lado, la corriente de un enchufe de la casa es totalmente diferente: cambia de positivo a negativo 50 veces por segundo en Argentina (en algunos países, 60 veces por segundo). Esto se conoce como corriente alterna (CA)

Los enchufes suelen tener 3 pines. A es el Vivo: provee el voltaje que alterna entre positivo y negativo en relación al pin B que es llamado Neutro. Si un dispositivo tiene una falla, debería proteger ahogando el voltaje al pin C:



Tierra.

*La corriente alterna es fundamental en ciertos casos, como para distribuir electricidad a largas distancias. También es útil en motores y electrodomésticos. La mayoría de los circuitos electrónicos simples funcionan con corriente continua, ya que es más fácil de manejar.*

## Interruptor

Como su nombre lo indica, interrumpen o conectan terminales. Suelen ser utilizados por ejemplo para encender un dispositivo (conectando la fuente de alimentación) Pueden tener 2 terminales o 3 terminales. En el caso de 3 terminales, permiten conectar cada extremo al terminal del medio.

## Resistencia

Como su nombre lo indica, “resiste” el flujo de electricidad. Las resistencias tienen un código de barras y colores que se lee de la siguiente manera. Primero se ubica la banda plateada o dorada a la derecha. Esa banda indica la tolerancia plateada 10%, oro 5%.

La resistencia eléctrica se mide en ohms. El símbolo griego omega ( $\Omega$ ) se usa para indicar ohms. La letra K (or  $K\Omega$ ) quiere decir un kilo ohm que es 1,000

ohms y la letra M (MΩ) un mega ohm, que son 1,000,000 ohms

Se usa ohm porque Georg Simon Ohm fue capaz de definir la relación fundamental entre tensión eléctrica, corriente y resistencia en 1827. Esto se conoce como la Ley de Ohm.

Las dos primeras bandas usan esta codificación

Negro 0

Marrón 1

Rojo 2

Naranja 3

Amrillo 4

Verde 5

Azul 6

Violeta 7

Gris 8

Blanco 9

La tercera banda dice cuántos ceros agregar

Negro - No ceros

Marrón 0 1 cero

Rojo 00 2 ceros

Naranja 000 3 ceros

Amarillo 0000 4 ceros

Verde 00000 5 ceros

Azul 000000 6 ceros

Violeta 0000000 7 ceros

Gris 00000000 8 ceros

Blanco 000000000 ceros

Una resistencia marrón-rojo-verde tendría valores de 1-2 y cinco ceros, resultando en 1,200,000 ohms, o 1.2MΩ. Una resistencia naranja-naranja-naranja tendría valores de 3-3 y tres ceros, resultando en 33,000 ohms, o 33KΩ.

## **Diodo y LED**

Un diodo es un componente que permite que la corriente eléctrica fluya en un solo sentido. Y un LED es un diodo emisor de luz. Los LED son económicos, duran mucho, vienen con distintos colores y suelen usarse en proyectos electrónicos para indicar información: encendido del equipo por ejemplo. Como regla mnemotécnica, el terminal largo del LED debe recibir más voltaje positivo que el terminal corto. La diferencia de voltaje entre los terminales y la corriente no debe exceder la indicación del fabricante.

## **Potenciómetro**

Los potenciómetros son componentes que permiten variar la resistencia (y por lo tanto el voltaje y la corriente) Suelen ser utilizados por ejemplo para regular el volumen de un equipo de audio. Suelen tener tres contactos: los extremos son el divisor de voltaje y el contacto medio es la salida con la resistencia aplicada.

## Capacitor

Un capacitor es como una batería chica recargable. Debido a su tamaño se carga en una fracción de segundo. La capacitancia (capacidad de almacenamiento eléctrica) se mide en faradios. La corriente DC no fluye a través de un capacitor, sino que el voltaje se acumula muy rápido y permanece cuando la fuente es desconectada. En electrónica se suelen usar cerámicos (discos, sin polaridad) y electrolíticos (barriles, con polaridad)

## Relay

El concepto de switching es fundamental en la electrónica y por switching debe entenderse usar un flujo de electricidad para controlar otro. Los dispositivos digitales no podrían existir sin este principio. Un relay es un caso típico de switching donde se usa una corriente menor para controlar una corriente mayor. Por ejemplo: se envía una señal de 5V DC para habilitar el

paso de 220 AC Adentro del Relay comúnmente un coil genera atracción magnética y cierra dos contactos.

## Transistor

Este componente fue inventado en Bell Labs en 1948. Es capaz de interrumpir y habilitar el paso de electricidad como un relay, pero es más sensitivo y versátil. Un transistor es un semiconductor en términos que en ocasiones conduce electricidad. Los transistores tienen tres patas: emisor, base y collector. El collector recibe corriente, el base la controla y el emisor la envía. La resistencia interna varía de acuerdo a la potencia que se aplica a la base. Los transistores NPN se abren con voltaje positivo y los PNP con voltaje negativo a la base. Los transistores bipolares amplifican corriente, no voltaje. Una fluctuación pequeña en la base habilita un cambio grande de corriente entre emisor y receptor. Una limitación del transistor es que está apagado hasta que se prende, mientras que un relay puede estar siempre prendido hasta que se apaga.

*El transistor 2N2222 es el semiconductor más utilizado de la historia fue desarrollado por Motorola en 1962 y todavía se fabrica. Como la patente caducó, cualquier empresa lo puede fabricar.*

## Ampers-hora

Las baterías suelen indicar ah, lo cual quiere decir ampers por hora. Una batería de 9v es de 0.3 amp-hours, mientras envíe 100 mA. Una pila AA 2.2 amp-hours, mientras envíe lo mismo: 100 mA

Agregando una resistencia a un capacitor se logra que el capacitor tarde más en cargar y en consecuencia habilita la temporalidad. De este modo se van combinando los componentes electrónicos para formar circuitos complejos.

# ARDUINO

Arduino es una plataforma electrónica de código abierto desarrollada en Italia. Las placas Arduino y sus placas derivadas suelen tener una serie de pines de alimentación (3.3V, 5V, VIN, GND), comunicación (SDA, SCL, UART tx, UART rx, etc) y pines genéricos de entrada/salida tanto digitales como analógicos (D2, D3, A0, etc) Los pines digitales solo pueden recibir y transmitir 0 y 1, mientras que los analógicos pueden tener un rango de valores 0 a 5V con un mapeo de 0 a 1023 vía ADC. A través de estos pines es posible conectar por ejemplo un interruptor o bien un sensor de humedad, procesar esas lecturas y generar algún tipo de salida como encender una luz, mover un motor o publicar algo en Internet. Para indicarle al Arduino qué hacer se le suben una serie de instrucciones en lenguaje de programación C++ al microcontrolador de la placa. Luego al recibir alimentación empieza automáticamente a ejecutar dicho código.

## **PARTES DE UN CÓDIGO ARDUINO**

El código C++ para un Arduino consta de dos partes: setup y loop. La primera parte solo se ejecuta una vez y como su nombre lo indica contiene el “setup” o la configuración inicial. La segunda parte se ejecuta repetitivamente o bien en “loop”. Esto es importante para determinar adónde colocar qué partes de nuestro código.

Si lo que queremos programar solo se ejecuta una vez, puede estar íntegramente en el setup y la función loop quedar vacía o bien estar en loop con algún mecanismo (condicional o un subloop) a fin de que solo sea ejecutado una vez.

## VARIABLES Y CONSTANTES

```
int ledVerde=2;  
const float pi = 3.14;
```

La primera línea quiere decir que ledVerde es de tipo Integer y se le asigna el valor 2. La segunda que pi es una constante float con valor 3.14

Todas las líneas en el código Arduino tienen que finalizar con punto y coma.

## PINES

```
pinMode(2, OUTPUT);
```

Con `pinMode` se indica primero el número de pin y luego si el pin referenciado es de entrada o de salida. El número de pin puede indicarse directamente en la función - `pinMode(2,OUTPUT)`; - pero conviene tenerlo definido en una variable para facilitar modificaciones. Esta definición se usa por ejemplo para conectar un LED.

Si se quiere por ejemplo leer un interruptor la definición del pin indicará `INPUT` o `INPUT_PULLUP`, que agrega una resistencia de aproximadamente 20k ohm entre el PIN y 5V.

## PWM

Dado que Arduino no puede variar la tensión de salida de sus pines, es necesario usar una técnica llamada Modulación por Ancho de Pulso o PWM para, por ejemplo desvanecer suavemente la luz de un LEDs. PWM consigue que un pin de salida varíe rápidamente su tensión de alto a bajo (entre 5 y 0V) durante un periodo fijo de tiempo. Este cambio de tensión se realiza tan rápido que el ojo humano no puede verlo. Cuando la tensión del pin cambia rápidamente de alto (HIGH) a bajo (LOW) durante un periodo fijo de tiempo, es como si se pudiese variar el nivel de la tensión de ese pin. El porcentaje de tiempo que un pin está en estado HIGH con respecto al tiempo total (periodo) se conoce con el nombre de relación cíclica. Cuando el pin esta en HIGH durante la mitad del periodo y en estado LOW durante la otra mitad, la

relación cíclica es del 50%. Una relación cíclica baja hace que el diodo LED tenga una luz mucho mas tenue que con una relación cíclica más alta. Los pines que pueden ser utilizados para PWM están indicados con el prefijo ~

```
analogWrite(9,127);
```

Esta instrucción para PWM indica primero el pin de salida y luego el duty cycle (0 a 255)

## COMENTARIOS

Utilizando dos barras iniciales se definen comentarios en el código, lo cual es ideal para documentar lo que se hace. Por ejemplo:

```
// inicialización de los pines
```

## MONITOR SERIAL

El Arduino puede no estar conectado a ningún dispositivo de salida como

pantallas o actuadores y aún así quizás necesitamos leer alguna variable sobre la ejecución. Para este fin podemos usar el monitor serial.

```
Serial.begin(115200);  
Serial.printl
```

La primera línea indica que se abre una línea de comunicación serial a 115200 de velocidad. La segunda línea enviará desde el Arduino al Arduino IDE (software instalado en la PC, Mac o Linux), la leyenda Hola mundo.

## CONDICIONALES

Los condicionales son sectores del código que solo se ejecutan cuando se cumple una condición. Es importante notar que la comparación de igualdad se hace con dos símbolos de igual, para diferenciarlo de la asignación. La condición en el ejemplo es: cuando el estado del Relay sea HIGH, cambiarlo a LOW y viceversa.

```
if (relayState==HIGH){  
    digitalWrite(relay, LOW);  
}  
else{  
    digitalWrite(relay, HIGH);  
}
```

## LOOPS

Independientemente de que el código principal de Arduino se ejecute en Loop una y otra vez, a menudo es necesario definir loops internos y esto se hace con la instrucción `while` y una condición especificada entre paréntesis.

```
while(relayState==HIGH){  
    // acá va el código a repetir  
}
```

## FOR

En el caso que sea necesario repetir un segmento de código iterando valores de un contador, es posible usar el siguiente código

```
for(int i=1; i<=10; i++){
// código que se ejecuta de 1 a 10
}
```

## FUNCIONES

Las funciones son ideales para reusar partes de código. En Arduino IDE se definen anteponiendo `function` y el retorno de la función se hace con la palabra clave `return`. En el ejemplo que sigue a continuación se muestra una función que reemplaza el símbolo de grado por la palabra “grados”.

```
function removeSpecial($str){
  $str=str_replace("°"," grados ",$str);
  return $str;
}
```

## ESP32

El ESP32 fue creado en 2016 por una empresa china que ya había ganado popularidad con el ESP8266, llamada Espressif Systems. A diferencia de

Arduino, que originalmente se desarrolló como una plataforma educativa basada, el ESP32 fue diseñado como un SoC (System on Chip) integrando en el mismo chip Wi-Fi y Bluetooth, además de un procesador de 32 bits, más memoria y periféricos. En aquel entonces, mientras que un Arduino Uno tradicional necesitaba hardware adicional para conectarse a una red, el ESP8266 y el ESP32 podían hacerlo directamente. La popularidad del ESP32 no surgió de una estrategia de marketing de Espressif, sino de la comunidad Maker: el bajo precio, el soporte para Arduino IDE y los clones hicieron que el ESP32 terminara convirtiéndose en una de las plataformas de hardware abierto más utilizadas.

*Es importante notar que también existen Arduinos con WiFi como UNO R4 WiFi, UNO Q e incluso un Arduino Nano ESP32*

# UART



UART - acrónimo de Universal Asynchronous Receiver/Transmitter = es uno de los protocolos de comunicación más utilizados entre dispositivos. Puede encontrarse por ejemplo en la conexión de microprocesadores con módulos de GPS y Bluetooth o bien en dispositivos diversos de hardware, en tanto mecanismo de diagnóstico o accesos de consola. UART es un protocolo de comunicación basado en transmisión serial asincrónica con velocidad configurable. Asincrónica significa que no existe una señal de reloj compartida entre emisor y receptor. El transmisor genera el flujo de bits con su propio reloj y el receptor usa el suyo para la llegada de datos. La sincronización se mantiene configurando el mismo baud rate en ambos dispositivos.

## SERIAL Y PARALELO

En una transferencia digital paralela, todos los bits viajan simultáneamente desde el origen al destino gracias a que existen múltiples líneas entre transmisor y receptor. En la comunicación serial como UART, en cambio, los datos se envían bit a bit usando una sola línea. Y dado que la comunicación suele ser bidireccional se usan dos cables. El TX del primer dispositivo debe conectarse al RX del segundo, y viceversa. El UART transmisor recibe datos paralelos del CPU (microprocesador o microcontrolador) y los convierte en serie. Esa trama serial viaja al UART receptor, que la vuelve a convertir en datos paralelos para entregarlos al CPU. UART organiza la transmisión en paquetes. Cada paquete incluye un bit de inicio, una trama de datos, un bit de paridad y uno o más bits de parada.

# COMANDOS AT

Los comandos AT son un lenguaje de texto simple creado en 1981 para los módems Hayes.

*Las famosas letras AT vienen de Attention.*

Ejemplo:

```
AT (¿Estás ahí?)  
OK (Sí)
```

La idea de los comandos AT era básica y elegante, el módem siempre está en uno de dos modos: Modo datos (transparente, todo lo que llega se envía a la línea telefónica). Modo comando (permite cambiar configuraciones, etc) Los comandos AT suelen enviarse a través de conexiones seriales UART y se encuentran en módems, módulos bluetooth, drones, incluso en satélites Starlink o módulos ESP32.

Ejemplos de comandos AT para módulos ESP32:

```
Escanear redes AT+CWLAP  
Enviar datos TCPAT+CIPSEND=tamaño
```

La simplicidad, legibilidad y compatibilidad del AT le dio el status de “esperanto” de los dispositivos de comunicación embebidos.

## UART HACK

Independientemente del uso estándar para comunicar sensores por ejemplo, lo cual suele estar bien documentado, es posible encontrar UART en diversos dispositivos en tanto acceso de diagnóstico y punto de entrada para cargar firmware modificado. En ocasiones las placas PCB tienen rotulada directamente la conexión UART como tal o bien indicando TX, RX. Otras veces se puede inferir utilizando un multímetro. Se coloca el multímetro en función continuidad y se busca el GND puentando hacia algún sector del PCB que claramente sea GND. Luego, la distinción entre los pines TX y RX se lleva a cabo colocando el multímetro en modo DCV, un contacto al GND y el otro al pin a identificar. Si es RX se mantendrá alto y similar a VCC, a menos que le estemos enviando datos. Si es TX debería fluctuar. Independientemente de la localización de los pines, habrá luego que probar el baud rate, bit de paridad y control de flujo. Con los datos inferidos, un adaptador USB a TTL y un terminal ya se podrá establecer la comunicación UART a ese dispositivo.

# LORA

LoRa fue creado en Francia en el año 2009 para permitir comunicaciones lentas a largas distancias usando ondas de radio de alta frecuencia, con un consumo mínimo. Es una capa física de modulación que junto con el protocolo de red LoRaWAN, crea una arquitectura completa de comunicación para IOT.

*Muchas de las tecnologías maker provienen de países que no son Estados Unidos o China. LoRa fue desarrollado en Francia. Arduino en Italia, Raspberry Pi en Inglaterra y Prusa en República Checa.*

## CHIRP

LoRa no utiliza la modulación de desplazamiento de frecuencia (FSK) tradicional, sino una modulación propia conocida como CSS (Chirp Spread Spectrum) donde la señal de datos se distribuye sobre un ancho de banda mucho mayor que el necesario para la información pura. Esto hace que la señal sea muy robusta contra el ruido y la interferencia. LoRa opera en las bandas de radiofrecuencia ISM (Industrial, Científica y Médica) sin licencia, las cuales varían según la región y están sujetas a regulaciones locales:

Europa 863-870MHZ

Norteamérica 902-928MHZ

Asia/Australia 915/928 y 920/923 MHZ

Dentro de estas bandas, se definen múltiples canales. Por ejemplo, en Europa,

se suelen usar canales base en 868.1 MHz, 868.3 MHz, 868.5 MHz.

Los módulos LoRa suelen comunicarse con Arduino y Raspberry por comandos AT vía UART. A modo de ejemplo: AT+SEND=7,10,HOLA MUNDO Indica enviar 10 caracteres Hola Mundo a la dirección 7

## MESHTASTIC

Iniciativas como Meshtastic utilizan LoRa para construir redes de comunicación off-grid, sin depender de Internet ni de infraestructura celular. Cada dispositivo funciona como un nodo de la red y puede retransmitir mensajes de otros nodos, formando una malla que permite cubrir distancias mucho mayores que las que alcanzaría una comunicación directa. Un mensaje puede, por ejemplo, viajar desde un dispositivo a otro, ser retransmitido por varios nodos intermedios y finalmente llegar a su destinatario.

Meshtastic incorpora mecanismos de cifrado para proteger el contenido de los mensajes donde los nodos intermedios pueden participar en el encaminamiento sin necesidad de conocer el contenido de una comunicación cifrada. Esto permite una arquitectura interesante desde el punto de vista de la autonomía tecnológica: podemos construir una red de mensajería que siga funcionando aunque se caiga Internet y la telefonía celular.

*Es importante notar que aunque el contenido pueda estar cifrado de extremo a extremo, la propia existencia de las transmisiones, sus características radioeléctricas y cierta información necesaria para el funcionamiento de la red pueden revelar metadatos.*

# SISTEMA BINARIO

*“Od glifulwkdg hv sursruflrqdo do sholjur”*

*Mhdq Edxgurlloodug*

*La dificultad es proporcional al peligro*

*(Jean Baudrillard)*

Un sistema binario en esencia es un sistema de numeración en el que los números son representados utilizando 0 y 1. Esta simplicidad lo hace ideal para trabajar con computadoras que a su vez están basadas en la electrónica usando switches para almacenar y manipular archivos. A pesar de la simplicidad del sistema binario, con 0 y 1 es posible representar cualquier tipo de datos: texto, imágenes y sonido.

0 es 0

1 es 1

2 es 10

4 es 100

5 es 101

## BINARIOS A DECIMAL

1 0 1

-----

4 2 1

Sumamos los valores que tienen 1:  $4+1=5$  (el 2 no se suma ya que tiene valor 0)

Otro ejemplo

1010010

1 0 1 0 0 1 0

-----

64 32 16 8 4 2 1

Sumamos los valores que tienen 1:  $64 + 16 + 2 = 82$

## DECIMAL A BINARIO

Se divide por 2 y si hay resto se indica 1. En caso contrario 0. Luego se reconstruye de manera inversa.

Ejemplo para el número 5

$5 / 2 = 2$  Residuo: 1

$2 / 2 = 1$  Residuo: 0

$1 / 2 = 0$  Residuo: 1

Luego reconstruimos a la inversa 101

Ejemplo para 6

$6/2 = 3$  Residuo 0

$3/2 = 1$  Residuo 1

$1/2 = 0$  Residuo 1

Reconstruimos a la inversa 110

Ejemplo con 11

$11 / 2 = 5$  Residuo: 1

$5 / 2 = 2$  Residuo: 1

$2 / 2 = 1$  Residuo: 0

$1 / 2 = 0$  Residuo: 1

Reconstruimos 1011

# LINUX

La línea de comandos de Linux es una interfase de texto con la computadora. Shell, terminal y console son algunos sinónimos de línea de comandos. Aprender línea de comandos de Linux es importante por un lado en la creación de dispositivos con Linux. Pero más allá de esa aplicación concreta la línea de comandos existe desde los años setenta, usa mínimos recursos y aún se encuentra en funcionamiento más o menos con la misma forma. Los entornos gráficos por su parte cambian a un ritmo donde hay poca continuidad y relación.

*Saber línea de comandos permite por reutilizar cualquier computadora que pueda ejecutar Linux y automatizar tareas complejas combinando distintos comandos en un shell script.*

## ACCESO A LA TERMINAL

Cualquier computadora con Linux tiene línea de comandos. La forma de acceso puede variar pero en general es posible clickear un ícono en el entorno gráfico o bien presionar la combinación de teclas CTRL-ALT-T. En el caso de un Raspberry sin entorno gráfico, la placa directamente bootea mostrando la línea de comandos.

### **pwd**

Es la abreviación de Print working directory. Nos da el path actual.

**cd / y cd home**

Con Change Directory, nos movemos al directorio root / y luego al home

**cd ~/Desktop**

Con este prefijo que quiere decir home, nos movemos desde el home a Desktop

**mkdir /home/Desktop/test**

Con el comando Make Directory creamos la carpeta test

**ls**

Con este comando (list) listamos los contenidos del directorio actual

**ls > listado.txt**

Con este comando podemos redireccionar la salida del list a un archivo. La redirección es una función muy potente que suele ser utilizada en los shell scripts

**cat listado.txt**

Con este comando podemos ver el contenido del archivo listado.txt Cat viene de conCATenar

**cat test1.txt test2.txt**

Este comando concatena test1 y test2 y los muestra en pantalla

**echo “Hola mundo”**

Este comando imprime lo que está entre comillas

**Wildcards**

(“?”) se usa para indicar cualquier carácter dentro del nombre de archivo. (“\*”) se usa para indicar 0 o más caracteres

**cat test?.txt**

Este comando por lo tanto imprimirá test1.txt y test2.txt

**Flecha Arriba, flecha abajo y TAB**

Tocando las flechas la línea de comandos va iterando por los comandos previos ejecutados. Tocando TAB es posible completar con información de los archivos del directorio actual. Ejemplo si ponemos cat te y tocamos TAB, va a completar cat test

**less textoLargo.txt**

Con este comando es posible paginar un texto largo. Se llama less porque antes había uno llamado More que solo permitía avanzar en la paginación.

**Case**

Linux es case sensitive. Esto quiere decir que UNO.TXT y uno.txt son dos archivos diferentes

**mv test.txt directorioa**

Este comando mueve test.txt al directorio

### **cp archivo.txt backup.txt**

Este comando copia un archivo con otro nombre a la misma ubicación o bien a otra ubicación

### **rm test.txt**

Este comando borra (remueve) test.txt. Puede ser utilizado también para directorios

### **wc -l test.txt**

Este comando word count permite contar palabras. Con el indicador -l cuenta las líneas

### **ls ~ | wc -l**

Este comando permite enviar la salida de un comando como la lista de archivos como entrada a otro comando (contar la cantidad de líneas)

### **man uniq**

Este comando permite consultar el manual. En este caso de uniq

### **sort test.txt | less**

Este comando ordena el contenido de test.txt y lo pagina

### **sudo cat /etc/shadow**

Este comando permite ejecutar con los privilegios de super user el cat Sudo suele utilizarse para instalar nuevo software, como por ejemplo `sudo apt-get XXXX` ¿Por qué usar sudo en lugar del usuario root? Para evitar errores que pudieran desencadenar en la generación de problemas para el SO.

### **Archivos ocultos**

Para ocultar un archivo basta con ponerle un `.` inicial. Esto se usa para mantener ocultos archivo de sistema por ejemplo que podrían interferir con archivos de usuario en ciertas carpetas. Con `ls -a` es posible listar todos los archivos, incluso los ocultos

# PROGRAMACIÓN

Para este anexo fue seleccionado el lenguaje de programación Python. Python es un lenguaje interpretado, esto quiere decir que los programas se pueden escribir con un editor de textos y luego se interpretan para ser ejecutados. Es un lenguaje sencillo de aprender, flexible, multiplataforma y con muchísimos packages para extender sus capacidades. Hay packages de computo científico, graficación y machine learning, por ejemplo.

¿Qué sentido tiene aprender a programar en Python o bien otro lenguaje cuando un LLM resuelve prácticamente cualquier asignación? Por un lado aprender a programar es formativo, desarrolla nuestras capacidades y nuestra comprensión del mundo que habitamos. Por otro lado, incluso cuando obviamente vamos a utilizar LLMs para la programación, nos da una perspectiva diferente, capacidades de auditar y mayor profundidad en los requerimientos.

## HOLA MUNDO

```
print("hola mundo")
```

Imprime en pantalla la cadena entre paréntesis.

## COMENTARIOS

Para comentar un texto se inicia la línea con #

```
# Esto sirve para explicar el código
```

## VARIABLES

```
contador=3
```

Asigno 3 a la variable contador

## SUMA Y MULTIPLICACIÓN

```
contador=contador+1 # suma  
  
contador=contador*2 # multiplicación
```

## SUBCADENAS

```
texto="hola mundo"  
  
print (texto[2:6])
```

Imprime la subcadena "la m".

*En programación se suele empezar a contar desde 0, de modo que la posición 0 es la letra h.*

## INGRESO CON TECLADO

Imprime cuál es tu nombre y lo almacena en la variable nombre.

```
nombre=input("¿Cual es tu nombre? ")
```

## FUNCIONES MATEMÁTICAS

```
from math import sin  
  
print(sin(45))
```

Imprime el seno de 45 grados.

## GRÁFICOS

Para graficar es necesario instalar un paquete desde línea de comandos

```
pip install matplotlib
```

Y luego vamos a crear un archivo de texto llamado plot.py con este contenido



```
import matplotlib.pyplot as plt

x = [ 1 , 2 , 3 , 4 , 5 , 6 , 7 , 8 , 9 , 10 ]

y = [ 5 , 2 , 4 , 4 , 8 , 7 , 4 , 8 , 10 , 9 ]

plt.plot( x , y)
plt.xlabel( 'Tiempo')
plt.ylabel('Unidades')
plt.show()
```

Solo resta ejecutar con python plot.py

## CONDICIONALES

El siguiente ejemplo el contenido de la variable a y b, determinando si es mayor, menor o igual.

```
a=5
b=8
if a>b:
    print("A es mayor que b")
else:
    print("A no es mayor que b")
if a==b:
    print("A es igual a b")
```

## ARRAYS

```
myArray=[1,1.2,3.4]

largo = len(myArray)

print(myArray[1])

print("Largo: "+str(largo))
```

# FOR LOOP

```
motos = [ 'Ducati' , 'Gilera', 'Honda' ]  
  
for moto in motos:  
  
    print(moto)
```

# WHILE

```
m = 10  
  
while m>5:  
    print(m)  
    m=m-1
```

# FUNCIONES

```
def suma (x,y):
    return x + y

print(suma(2,3))
print(suma
```

Si hace falta devolver más de un valor, se separan por comas:

```
return suma, media
```

## ARCHIVOS

```
# graba
f= open ("archivo.txt" , "w" )
f.write("Test")
f.close()

# lee
f= open ("archivo.txt" , "r" )
data = f.read()
print(data)
f.close()
```

Graba un archivo con el nombre archivo.txt y contenido "Test" y luego abre el archivo e imprime el contenido.

## RANDOM

```
import random
print (random.randint(1,10))
```

## SCRAPPEAR PÁGINA WEB

```
import urllib.request

link = "https://unSitioABC.com/?page=17"
f = urllib.request.urlopen(link)
myfile = f.read()
print(myfile)
```

# INTERNET

ARPA (Advanced Research Projects Agency) surgió durante la Guerra Fría como respuesta al lanzamiento del satélite soviético Sputnik, juntando en sus laboratorios matemáticos, ingenieros, psicólogos y programadores. A finales de los años sesenta, ARPA buscaba un sistema de comunicación resistente a fallos, incluso si partes de la red eran destruidas. Así nació ARPANET conectando cuatro universidades. La información en ARPANET se dividía en fragmentos independientes que viajaban por rutas variables y se re-ensamblaban en el destino. ARPANET evolucionó adoptando TCP/IP en 1983 y ese fue el origen de lo que hoy se conoce como Internet.

## TCP-IP

Internet se basa en la suite de protocolos TCP/IP. TCP (Transmission Control Protocol) asegura que los paquetes lleguen completos, en orden y sin errores, manejando también el control de flujo. IP (Internet Protocol) se ocupa del enrutamiento y el direccionamiento: cada dispositivo tiene una dirección única (como 192.0.2.1) que indica cómo encaminar los datos.

Las direcciones IP incluyen rangos privados como 10.0.0.0/8, 172.16.0.0/12 y 192.168.0.0/16 para redes internas.

El bloque 127.0.0.0/8 es para loopback: los paquetes enviados ahí vuelven a la misma máquina (comúnmente 127.0.0.1 para pruebas locales).

*0.0.0.0 representa “todas las interfaces” o una dirección indefinida, y 255.255.255.255 sirve para broadcasts en la red local.*

Las máscaras de subred, como 255.255.255.0, definen qué bits de la IP corresponden a la red y cuáles al host. Los “255” (bits en 1) marcan la porción de red, mientras que los ceros indican hosts. Esto permite a los routers decidir rápidamente si un paquete debe enrutarse localmente o hacia Internet.

*El agotamiento de direcciones IPv4 impulsó IPv6, con direcciones de 128 bits como 2001:0db8::1*

## DNS

Gracias a TCP/IP, cualquier máquina puede comunicarse con otra máquina en cualquier parte del mundo. Con el crecimiento de Internet, memorizar direcciones IP numéricas se volvió inviable. El DNS (Domain Name System) resolvió este problema con un directorio distribuido: al ingresar un dominio como “ejemplo.com”, una cadena de servidores lo traduce a su IP correspondiente. Otras tecnologías completan el ecosistema: IMAP, POP y SMTP para el correo. HTTP/HTTPS para facilitar la solicitud y entrega de documentos web. Al acceder a un sitio web, se inicia un proceso secuencial: consulta DNS para obtener la IP, envío de un paquete SYN para el handshake TCP, negociación de TLS para seguridad, solicitud HTTP/HTTPS, y recepción

de bytes que el navegador renderiza.

*El 29 de octubre de 1969 se intentó enviar el primer mensaje entre la UCLA y el Stanford Research Institute. El ingeniero Charley Kline quería transmitir "LOGIN", pero el sistema colapsó después de las letras "L" y "O". Este "LO" se convirtió en el primer mensaje de Internet.*

ARPA/DARPA también financió investigaciones sobre interacción hombre-máquina, reconocimiento de patrones, procesamiento del lenguaje, planificación y sistemas capaces de asistir en la toma de decisiones. Su Information Processing Techniques Office, creada en 1962 y fue particularmente importante para el desarrollo temprano de la IA.

# PRIVACIDAD

*"Nosotros no espiamos... Ellos le dicen telemetría"*

*Jean-Baptiste Kempf*

*Conferencia Nerdearla Argentina 2026*

Los gobiernos protegen comunicaciones militares y diplomáticas; las empresas resguardan secretos comerciales, pero también las personas necesitan preservar conversaciones, archivos y datos. Es posible esconder la existencia de un mensaje mediante tinta invisible o esteganografía, modificar su representación o transformar su contenido para que resulte incomprensible sin una clave.

## CODIFICACIÓN

Codificar consiste en cambiar la representación de la información para poder almacenarla o transmitirla: ASCII convierte caracteres en números y Base64 representa datos binarios mediante texto. Como las reglas son públicas y no requieren un secreto, la codificación no protege el contenido.

## SUSTITUCIÓN

Reemplazar cada letra, palabra o símbolo por otro según una regla acordada. En el cifrado César, por ejemplo, cada letra se desplaza una cantidad fija de posiciones dentro del alfabeto. La frase es ilegible a simple vista, pero su protección es débil ya que conserva regularidades del idioma, como la frecuencia con que aparecen determinadas letras y combinaciones. Un adversario no necesita conocer la regla exacta si puede reconstruirla

analizando suficientes mensajes.

## ENCRIPCIÓN

Durante siglos, la criptografía dependió de claves compartidas donde emisor y receptor tenían que conocer previamente el mismo secreto. El problema consiste en cómo distribuir la clave: si se debe enviar por el mismo canal que se intenta proteger, puede ser interceptada. La criptografía de clave pública, desarrollada durante la década de 1970, propuso una solución radical: separar la clave en dos partes matemáticamente relacionadas. Una clave pública puede difundirse sin restricciones, mientras que una clave privada permanece bajo el control de su propietario.

Los números primos adquirieron especial importancia en algunos de estos sistemas, particularmente en RSA, presentado por Ronald Rivest, Adi Shamir y Leonard Adleman. Para generar sus claves se eligen dos números primos grandes y se multiplican. Hacer esta multiplicación es fácil, pero recuperar los factores primos a partir del producto es costoso computacionalmente si los números son muy grandes. La clave pública contiene información derivada de ese producto, mientras que la clave privada depende de los factores originales.

## PGP

En 1991, el programador estadounidense Philip Zimmermann publicó PGP - Pretty Good Privacy - . Su objetivo era poner criptografía fuerte al alcance de las personas comunes, especialmente para proteger archivos y correo electrónico frente a la vigilancia estatal, corporativa o criminal. Zimmermann entendía la privacidad como una condición necesaria para la libertad personal, y distribuyó el programa gratuitamente.

La circulación internacional de PGP provocó una investigación del gobierno de Estados Unidos porque en ese momento, el software criptográfico fuerte tenía restricciones de exportación similares a las aplicadas sobre material militar. Zimmermann respondió de una manera creativa: en 1995 publicó, a través de MIT Press, PGP Source Code and Internals, un libro enorme cuyo contenido era el código fuente de PGP. Exportar ese mismo código en formato digital estaba restringido, pero los libros impresos podían circular internacionalmente. Una vez fuera de Estados Unidos, las páginas se escaneaban con OCR, se reconstruía el código fuente y se volvía a compilar el programa. La investigación contra Zimmermann se cerró en enero de 1996 sin presentar cargos.

PGP funciona como un criptosistema híbrido. Cuando una persona cifra un mensaje, el programa genera una clave de sesión aleatoria y utiliza un algoritmo simétrico para cifrar el contenido. Luego cifra esa pequeña clave de sesión con la clave pública del destinatario. Al recibir el archivo, el

destinatario utiliza su clave privada para recuperar la clave de sesión para descifrar el mensaje.

PGP también permite crear firmas digitales. En lugar de ocultar el mensaje, la firma produce una huella criptográfica que se procesa con la clave privada del autor. Cualquier persona que posea su clave pública puede comprobar que el contenido no fue alterado y que la firma corresponde a esa clave. Por eso se dice que PGP ofrece confidencialidad, integridad y autenticidad.

La complejidad de PGP está en la administración de las claves: verificar a quién pertenece una clave pública, proteger la clave privada, conservar copias de respaldo y revocar una clave comprometida.

## PGP

GPG (GNU Privacy Guard) es una implementación de OpenPGP. Permite cifrar archivos y mensajes, firmarlos digitalmente y administrar claves públicas y privadas.

En muchas distribuciones Linux ya viene instalado. Para comprobarlo:

```
gpg --version
```

En Debian, Ubuntu y derivados puede instalarse con:

```
sudo apt install gnupg
```

## Crear un par de claves

```
gpg --full-generate-key
```

GPG solicita el tipo y tamaño de clave, su período de validez y una identidad compuesta normalmente por el nombre y la dirección de email. También pide una contraseña para proteger la clave privada.

## Consulta de claves

```
gpg --list-keys
```

## Exportar una clave pública

```
gpg --armor --export alguien@example.com > public-key.asc
```

El archivo `public-key.asc` puede enviarse a cualquier persona que quiera cifrar información para nosotros o verificar nuestras firmas.

## Importar la clave pública de otra persona

```
gpg --import public-key.asc
```

## Firmar un mensaje

```
gpg --clearsign message.txt
```

Esto genera `message.txt.asc`, que contiene el texto original legible junto con una firma digital. La firma permite comprobar que el contenido no fue modificado y que fue firmado con la clave privada correspondiente.

## Para verificar la firma

```
gpg --verify message.txt.asc
```

## Cifrar para un destinatario

Primero debemos tener importada su clave pública. Luego:

```
gpg --encrypt --armor --recipient bandini@example.com message.txt
```

El archivo queda cifrado con la clave pública del destinatario y solamente puede descifrarse utilizando su clave privada.

## Para descifrar

```
gpg --decrypt message.txt.asc
```

Por ejemplo, si Juan quiere enviarle un mensaje secreto a Pedro, Juan necesita la clave pública de Pedro para cifrarlo. Pedro utiliza luego su clave privada para recuperarlo. Si Pedro quiere demostrar además que un mensaje fue realmente escrito por él, puede firmarlo con su clave privada y cualquiera que posea su clave pública podrá verificar la firma.

# ESTEGANOGRAFÍA

La esteganografía consiste en ocultar información dentro de otro archivo de manera que la existencia del mensaje pase inadvertida. A diferencia del cifrado, que hace ilegible un mensaje pero deja evidente que existe información protegida, la esteganografía intenta ocultar el propio mensaje. Una técnica clásica consiste en modificar los bits menos significativos de los píxeles de una imagen sin producir cambios perceptibles para el ojo humano.

Un ejemplo sencillo en Linux es utilizar steghide para esconder un archivo de texto dentro de una imagen JPEG:

```
# Instalar steghide
sudo apt install steghide

# Ocultar mensaje.txt dentro de foto.jpg
steghide embed -cf foto.jpg -ef mensaje.txt

# Extraer el mensaje
steghide extract -sf foto.jpg
```

## DETECTAR DISPOSITIVOS QUE NOS ESPÍAN

En el capítulo sobre Cultura Maker se plantea el principio de desconfianza sobre dispositivos que nos vigilan. Esto no es algo teórico, sino concreto. Once millones de SmartTVs Vizio E-Series y M-Series de 2014 a 2016 sampleaban pixels de la pantalla para generar un fingerprint y lo matcheaban contra una base de datos para determinar qué veían los usuarios. Vizio combinó esta información con IP, códigos postales e info demográfica para vender a advertisers. Samsung con su serie UE46ES8000 enviaba capturas del micrófono de las Tvs por el port 443/TCP a vendors.

Asistentes personales como Amazon Echo grababan conversaciones del ambiente y las enviaban a Amazon S3 buckets. Google por su parte vendía

Nest Secure guard system on un micrófono no documentado y juguetes como My Friend Cayla tenían Bluetooth y micrófonos sin seguridad que podían ser utilizados para monitorear el ambiente así como para usar el juguete como un parlante.

Para detectar datos de telemetría y comunicaciones sospechosas de nuestros dispositivos podemos llevar a cabo el siguiente procedimiento.

### **Identificar los dispositivos de la red local**

```
sudo arp-scan --interface=wlan0 192.168.1.0/24
```

### **Observar el tráfico en 192.168.1.50 (ejemplo)**

```
sudo tcpdump -i wlan0 host 192.168.1.50 -nn
```

### **Aislar las consultas DNS**

```
sudo tcpdump -i wlan0 host 192.168.1.50 and port 53 -nn
```

### **Observar las conexiones HTTPS/TLS**

```
sudo tcpdump -i wlan0 host 192.168.1.50 and port 443 -nn
```

### **Análisis más detallado**

```
sudo tcpdump -i wlan0 host 192.168.1.50 -w device.pcap
```

# BITCOIN

Bitcoin es un sistema de moneda digital descentralizada propuesto en el año 2008 por Satoshi Nakamoto. El objetivo fundamental es permitir transferencias de valor entre personas sin depender de bancos, gobiernos o intermediarios que validen las operaciones. Desde algún punto de vista Bitcoin no fue algo totalmente nuevo, sino más bien una combinación inteligente de tecnologías existentes: criptografía, redes peer-to-peer y mecanismos de consenso.

Las especulaciones sobre la identidad de Satoshi Nakamoto apuntaron a distintas figuras del mundo de la criptografía y el software, entre ellas Hal Finney, receptor de la primera transacción de Bitcoin; Nick Szabo, creador de Bit Gold; Adam Back, inventor de Hashcash; y Dorian Nakamoto, señalado por una investigación periodística. También fue señalado Craig Wright, aunque sus afirmaciones fueron ampliamente cuestionadas y no logró demostrar que controlara las claves criptográficas asociadas a Satoshi.

*Otras hipótesis sostienen que Nakamoto pudo haber sido un pequeño grupo de desarrolladores y no una sola persona. Hasta hoy la identidad real del creador de Bitcoin sigue siendo desconocida.*

Las transacciones de Bitcoin se registran en una base de datos pública y distribuida conocida como el blockchain. En lugar de una computadora central que conserva el registro, múltiples nodos mantienen copias independientes y verifican que las transacciones cumplan las reglas. Este registro permite demostrar matemáticamente quién tiene la capacidad de

gastar fondos.

El control de los fondos se lleva a cabo con criptografía de clave pública. Cada usuario tiene una clave privada que le permite firmar digitalmente sus transacciones. La red puede verificar esa firma sin conocer la clave privada. “Tener” bitcoins significa disponer de las claves necesarias para autorizar su transferencia. No hay que almacenar ningún bien físico – oro, papeles – sino una clave.

El principal problema de otras criptomonedas solía ser cómo resolver el doble gasto, es decir cómo impedir que una misma unidad digital pueda utilizarse simultáneamente en dos pagos, sin recurrir a una autoridad central. Bitcoin lo resuelve con un mecanismo llamado *Proof of Work*. Particulares, organizaciones o empresas tienen computadoras dedicadas a minería. Estas computadoras compiten utilizando su capacidad de cómputo para construir nuevos bloques de transacciones. Modificar retroactivamente la historia no es viable porque implica rehacer ese trabajo y superar al resto de la red, algo que se vuelve cada vez más costoso.

El protocolo también establece una política monetaria predecible. Como recompensa por asegurar la red se emiten nuevos bitcoins, pero esa emisión disminuye cada cuatro años mediante un proceso llamado *halving*. La cantidad máxima está limitada a 21 millones de bitcoins.

*En definitiva, Bitcoin reemplaza la confianza institucional por reglas*

*matemáticas, incentivos económicos y consenso distribuido.*

# NOTAS FINALES

La tercera edición ordena los segmentos teóricos, expande contenidos y agrega capítulos de referencia sobre privacidad y criptomonedas.

Quiero agradecer especialmente a los alumnos que se inscribieron en los talleres de Contracultura Maker y a los asistentes de las charlas. Estos espacios fueron el punto de partida para convertir mis notas en el material iniciático del libro. Gracias también a todos los que se tomaron un momento para realizar una devolución o referencia.

Roni Bandini

Buenos Aires, Argentina

# ÍNDICE

|                                             |     |
|---------------------------------------------|-----|
| Introducción .....                          | 1   |
| Maquinas, autómatas, robots y cyborgs ..... | 3   |
| Cultura Maker.....                          | 14  |
| Contracultura Maker.....                    | 17  |
| Inteligencia artificial .....               | 31  |
| Electrónica .....                           | 46  |
| Arduino .....                               | 56  |
| UART .....                                  | 65  |
| LORA .....                                  | 69  |
| Sistema binario .....                       | 73  |
| Linux .....                                 | 77  |
| Programación .....                          | 83  |
| Internet .....                              | 93  |
| Privacidad .....                            | 97  |
| Bitcoin .....                               | 108 |
| Notas finales .....                         | 112 |
| Indice .....                                | 114 |
| Bibliografía .....                          | 116 |

# **BIBLIOGRAFÍA Y REFERENCIAS**

Taylor, F. W.. The Principles of Scientific Management. Harper & Brothers, 1911. ISBN: 978-0486299889.

Shannon, C. E.. A Mathematical Theory of Communication. Bell System Technical Journal, 1948. DOI: 10.1002/j.1538-7305.1948.tb01338.x.

Russell, S.; Norvig, P.. Artificial Intelligence: A Modern Approach. Pearson, 2020. ISBN: 978-0134610993.

Situnayake, D.; Plunkett, J.. AI on the Edge. O'Reilly Media, 2022. ISBN: 978-1098120207.

Domingos, P.. The Master Algorithm. Basic Books, 2015. ISBN: 978-0465065707.

Platt, C.. Make: Electronics. Maker Media, 2015. ISBN: 978-1680450262.

Muslin, E.. Máquinas del siglo XX. Editorial MIR, 1974.

Standage, T.. The Mechanical Turk. Walker & Company, 2002. ISBN: 978-0802714309.

Posselt, E. A.. The Jacquard Machine Analyzed and Explained. John Wiley & Sons, 1902.

Editora Verde Olivo. Con nuestros propios esfuerzos. Editora Verde Olivo, 1992.

Shelley, M.. *Frankenstein; or, The Modern Prometheus*. Lackington, Hughes, Harding, Mavor & Jones, 1818.

Dick, P. K.. *Do Androids Dream of Electric Sheep?*. Doubleday, 1968. ISBN: 978-0345404473.

Dahl, R.. *The Sound Machine*. The New Yorker, 1949.

Roszak, T.. *The Making of a Counter Culture*. Doubleday, 1969. ISBN: 978-0520212270.

Gibson, W.. *Neuromancer*. Ace Books, 1984. ISBN: 978-0441569595.

Baudrillard, J.. *Simulacra and Simulation*. University of Michigan Press, 1994. ISBN: 978-0472065219.

Ashby, W. R.. *An Introduction to Cybernetics*. Chapman & Hall, 1956. ISBN: 978-0416683009.

Alonso, R.. *Elogio de la low-tech*. Fundación Telefónica / catálogo, 2015.

Simondon, G.. *El modo de existencia de los objetos técnicos*. Aubier, 1958.

ISBN: 978-2700700573.

Crawford, M. B.. Shop Class as Soulcraft. Penguin Press, 2009. ISBN: 978-0143117469.

Mumford, L.. Technics and Civilization. Harcourt, Brace and Company, 1934. ISBN: 978-0226550275.

Burroughs, W. S.. Naked Lunch. Olympia Press, 1959. ISBN: 978-0802132958.

Clynes, M. E.; Kline, N. S.. Cyborgs and Space. Astronautics, 1960. Referencia de artículo.

Čapek, K.. R. U. R. (Rossum's Universal Robots). Aventinum, 1920.

Weizenbaum, J.. ELIZA – A Computer Program for the Study of Natural Language Communication. Communications of the ACM, 1966. DOI: 10.1145/365153.365168.

Vaswani, A.; Shazeer, N.; Parmar, N.; Uszkoreit, J.; Jones, L.; Gomez, A. N.; Kaiser, Ł.; Polosukhin, I.. Attention Is All You Need. NeurIPS, 2017. arXiv:1706.03762.

McCarthy, J.; Minsky, M. L.; Rochester, N.; Shannon, C. E.. A Proposal for the Dartmouth Summer Research Project on Artificial Intelligence. Dartmouth

College, 1955. Documento histórico.

Rosenblatt, F.. The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain. Psychological Review, 1958. DOI: 10.1037/h0042519.

Rumelhart, D. E.; Hinton, G. E.; Williams, R. J.. Learning representations by back-propagating errors. Nature, 1986. DOI: 10.1038/323533a0.

Postman, N.. Technopoly: The Surrender of Culture to Technology. Alfred A. Knopf, 1992. ISBN: 978-0679745402.

Banzi, M.; Shiloh, M.. Getting Started with Arduino. Maker Media, 2022. ISBN: 978-1457186439.

Thompson, H. S.. The Great Gatsby (copying exercise described in Gonzo practice). F. Scott Fitzgerald / Charles Scribner's Sons, 1925.

Contracultura Maker

Versión 3.1

Roni Bandini

Buenos Aires, Argentina

Octubre de 2026

Licencia CC BY 4.0

Los contenidos de este libro se proporcionan con fines educativos, experimentales y de divulgación. El autor no garantiza que la información, los procedimientos, los códigos, circuitos, diseños o ejemplos estén libres de errores o sean adecuados para un propósito particular. La aplicación práctica de los contenidos y sus posibles consecuencias son responsabilidad de quien los utiliza. Se recomienda verificar la información y adoptar las medidas de seguridad y precauciones que correspondan antes de realizar cualquier implementación.